

## Contents

- [C++test Properties Guide for Three Parallel Scans](#)
  - [Purpose](#)
  - [1. A simple model](#)
  - [2. What maps from the current properties file](#)
    - [Active settings](#)
    - [Commented settings](#)
  - [3. Recommended files](#)
    - [MISRA scan](#)
    - [Flow Analysis scan](#)
    - [CERT C scan](#)
  - [4. Isolate the three Standard processes](#)
  - [5. Docker requirements](#)
  - [6. Customer acceptance checklist](#)
    - [Before execution](#)
    - [During execution](#)
    - [After execution](#)
  - [7. What the customer actually changes](#)

# C++test Properties Guide for Three Parallel Scans

---

## Purpose

---

This guide shows how to reuse the existing `cpptestcli.properties` knowledge when three C++test Standard 2023.1.1 processes run concurrently in one Docker container and publish to DTP 2022 (10.6).

The properties migration is not a product reconfiguration. Most settings are copied unchanged. The important changes are:

1. Give each scan its own completed properties file.
2. Mount those files read-only and select one with `-settings`.
3. Give each scan a distinct DTP project/build/session identity as appropriate.
4. Run each scan from a separate checkout or worktree so its `.cpptest` cache is private.
5. Keep reports and completion markers in separate output directories.

## 1. A simple model

---

Use three layers:

| Layer               | Contains                                                                                  | Changes per scan? |
|---------------------|-------------------------------------------------------------------------------------------|-------------------|
| Shared installation | C++test Standard 2023.1.1 binaries                                                        | No                |
| Per-scan properties | DTP connection, requested features, analysis configuration, compiler, DTP result identity | A few values      |
| Per-scan command    | Source checkout, compilation database, report directory, heap limit                       | Yes               |

Do not replace `/opt/parasoft/cpptest/cpptestcli.properties` when a job starts. Concurrent jobs could overwrite that shared file while another process is reading it. Instead, place the three job files in a protected directory:

```
/run/secrets/cpptest/  
├─ misra.properties  
├─ flow.properties  
└─ cert.properties
```

Each job explicitly selects its file, for example: `cpptestcli -settings /run/secrets/cpptest/misra.properties ....`

## 2. What maps from the current properties file

### Active settings

These active settings map directly into every per-scan file:

| Current purpose                   | Property                                                                                              | Recommended handling                                    |
|-----------------------------------|-------------------------------------------------------------------------------------------------------|---------------------------------------------------------|
| Accept the EULA                   | <code>parasoft.eula.accepted=true</code>                                                              | Copy unchanged                                          |
| Use the network License Server    | <code>cpptest.license.use_network=true</code>                                                         | Copy unchanged                                          |
| Select requested product features | <code>cpptest.license.network.edition</code> and <code>cpptest.license.custom_edition_features</code> | Copy the approved values unchanged                      |
| Connect to DTP                    | <code>dtp.url</code> , <code>dtp.user</code> , <code>dtp.password</code>                              | Copy into each protected file                           |
| Assign DTP results                | <code>dtp.project</code>                                                                              | Set for the intended DTP project                        |
| Publish results                   | <code>report.dtp.publish=true</code>                                                                  | Copy unchanged when publication is required             |
| Select analysis                   | <code>cpptest.configuration</code>                                                                    | Set for MISRA, Flow, or CERT                            |
| Select compiler model             | <code>cpptest.compiler.family=gcc_8-64</code>                                                         | Use when all projects build with the matching GCC model |

| Current purpose            | Property                                  | Recommended handling                                  |
|----------------------------|-------------------------------------------|-------------------------------------------------------|
| Label a DTP build          | <code>build.id</code>                     | Supply a stable CI build identifier                   |
| Distinguish a test session | <code>session.tag</code>                  | Supply a unique scan identifier                       |
| Increase console detail    | <code>console.verbosity.level=high</code> | Recommended for the demonstration and troubleshooting |

The existing file already contains the required knowledge. The migration mainly activates `build.id`, `session.tag`, and the compiler property, then gives each analysis its own configuration.

## Commented settings

Commented lines are reference options, not requirements. Leave them commented unless the current workflow uses them:

- HTML/PDF report details
- email delivery
- source-control authorship
- technical-support archives
- custom file encoding
- Flow Analysis data reuse
- usage reporting

Enabling every documented option makes the file harder to maintain and may introduce unnecessary paths or credentials. Start with the active minimum and add an option only when a job needs it.

### 3. Recommended files

---

Create the files from the sanitized repository template:

```
install -d -m 700 /run/secrets/cpptest
install -m 600 examples/customer-settings/cpptestcli.properties.example \
/run/secrets/cpptest/misra.properties
install -m 600 examples/customer-settings/cpptestcli.properties.example \
/run/secrets/cpptest/flow.properties
install -m 600 examples/customer-settings/cpptestcli.properties.example \
/run/secrets/cpptest/cert.properties
```

Replace every <PLACEHOLDER>. Do not commit completed files. An encoded password is still a credential.

The three files may share these values:

```
parasoft.eula.accepted=true
cpptest.license.use_network=true
cpptest.license.network.edition=custom_edition
cpptest.license.custom_edition_features=<APPROVED_FEATURE_LIST>

dtp.enabled=true
dtp.url=<DTP_URL>
dtp.user=<DTP_USER>
dtp.password=<ENCODED_DTP_PASSWORD>

cpptest.compiler.family=gcc_8-64
report.dtp.publish=true
console.verbosity.level=high
```

#### MISRA scan

```
dtp.project=<MISRA_DTP_PROJECT>
cpptest.configuration=c++test.dtp://<MISRA_CONFIGURATION_NAME>
build.id=<BUILD_ID>
session.tag=misra-<BUILD_ID>
```

#### Flow Analysis scan

```
dtp.project=<FLOW_DTP_PROJECT>
cpptest.configuration=c++test.dtp://Flow Analysis Standard
build.id=<BUILD_ID>
session.tag=flow-<BUILD_ID>
```

#### CERT C scan

```
dtp.project=<CERT_DTP_PROJECT>
cpptest.configuration=c++test.dtp://<CERT_C_CONFIGURATION_NAME>
build.id=<BUILD_ID>
session.tag=cert-<BUILD_ID>
```

Use the exact DTP configuration names available in the customer environment. If all results belong to one DTP project, `dtp.project` may be shared, but `session.tag` should still distinguish MISRA, Flow, and CERT. Before each run, CI should render `<BUILD_ID>` and any other dynamic placeholders into protected job-specific files.

Do not define the same setting in several places without a reason. For example, select the configuration in the properties file or with `-config`, not both. The examples in this guide keep configuration and compiler selection in the properties file.

## 4. Isolate the three Standard processes

---

C++test Standard stores `.cpptest` state and a lock below its current working directory. It does not support the Professional-only `-data` option.

Therefore, three concurrent processes must use three separate source checkouts or Git worktrees:

```
/work/misra/    source + compile_commands_merged.json + .cpptest/
/work/flow/     source + compile_commands_merged.json + .cpptest/
/work/cert/     source + compile_commands_merged.json + .cpptest/

/results/misra/report/
/results/flow/report/
/results/cert/report/
```

Do not run all three commands from the same checkout. Sharing that directory causes `.cpptest` lock contention even when report directories differ.

The repository launcher enforces the correct pattern:

```
RUN_ROOT=/results ./examples/customer-settings/run-one-analysis.sh \
  misra /work/misra /run/secrets/cpptest/misra.properties &
pid_misra=$!

RUN_ROOT=/results ./examples/customer-settings/run-one-analysis.sh \
  flow /work/flow /run/secrets/cpptest/flow.properties &
pid_flow=$!

RUN_ROOT=/results ./examples/customer-settings/run-one-analysis.sh \
  cert /work/cert /run/secrets/cpptest/cert.properties &
pid_cert=$!

set +e
wait "$pid_misra"; result_misra=$?
wait "$pid_flow"; result_flow=$?
wait "$pid_cert"; result_cert=$?
set -e

printf 'MISRA=%s FLOW=%s CERT=%s\n' \
  "$result_misra" "$result_flow" "$result_cert"
```

The launcher:

- changes directory to the selected source checkout;
- passes the selected properties file with `-settings`;
- analyzes `compile_commands_merged.json`;
- writes to a per-project report directory;
- preserves the existing 16 GiB Java maximum heap by default;
- creates a per-project completion marker after a completed analysis;
- treats exit code 137 as an incomplete run.

With `-fail`, a completed scan can return a nonzero code because findings violated policy. That is different from termination or an incomplete cache.

## 5. Docker requirements

---

Use one container with the C++test installation mounted or built once, plus isolated inputs and outputs:

```
services:
  cpptest:
    image: customer-cpptest-standard:2023.1.1
    volumes:
      - ./work/misra:/work/misra
      - ./work/flow:/work/flow
      - ./work/cert:/work/cert
      - ./results:/results
      - ./secrets/cpptest:/run/secrets/cpptest:ro
```

The three processes share binaries and network access, but they do not share writable analysis state.

The current command allows up to 16 GiB of Java heap per scan. Three scans can therefore request up to 48 GiB of Java heap, plus compiler, native C++test, filesystem cache, report, and operating-system overhead. Treat container and host sizing as a measured deployment decision, not as a fixed universal number.

For the first production-like test:

1. Start with sufficient host memory above the 48 GiB aggregate heap ceiling.
2. Set an explicit container memory limit and monitor peak usage with `docker stats`.
3. Reserve enough CPU for three useful concurrent processes. A CPU quota limits consumption but does not reserve exclusive CPU time.
4. Use dedicated host capacity or CPU pinning if strict performance isolation is required.
5. Keep DTP, License Server, source-control, and DNS endpoints reachable from the container.
6. Store result volumes on storage with enough free space and I/O throughput.

If memory pressure occurs, reduce `CPPTTEST_MAX_HEAP` based on measured project demand or increase container/host memory. Never hide exit code 137 or publish a partial `.cpptest` cache.

## 6. Customer acceptance checklist

---

### Before execution

- C++test Standard 2023.1.1 starts inside the target container.
- The configured GCC compiler identifier is available.
- Each DTP-hosted test configuration can be listed and opened.
- The properties files contain no unresolved placeholders.
- Completed properties files are protected and excluded from source control.
- Each job has a distinct checkout, report directory, and completion marker.
- `build.id` and `session.tag` produce the required DTP grouping.
- Container access to DTP and License Server is confirmed.

### During execution

- Three separate `cpptestcli` Java processes are visible at the same time.
- `docker stats` shows CPU and memory consumption for the single container.
- Each console identifies the expected test configuration, compiler, project, build, and session.
- No process reports a `.cpptest` lock held by another process.

### After execution

- All three processes reached analysis completion.
- Each output directory contains its own report and completion marker.
- Findings appear under the expected DTP project/build/session.
- A policy-findings exit is distinguished from infrastructure failure.
- No credentials appear in logs, reports, images, or repository files.

## 7. What the customer actually changes

---

The customer does not need to redesign the properties system. The practical changes are limited to:

1. Stop copying a job-specific file over the installation's shared `cpptestcli.properties`.
2. Maintain one protected `-settings` file per analysis profile.
3. Put configuration and compiler choices in that file consistently.
4. Activate meaningful `build.id` and `session.tag` values for DTP result separation.
5. Give every concurrent Standard process a separate checkout and writable `.cpptest` cache.
6. Give every process a separate report directory and completion marker.

Everything else in the existing properties file either maps directly or remains optional, commented reference material.

**Resources:** Public demonstration: <https://zuwasi.github.io/cpptest-parallel-automation-demo/>. Repository: <https://github.com/zuwasi/cpptest-parallel-automation-demo>. Implementation files: `examples/customer-settings/cpptestcli.properties.example`, `examples/customer-settings/run-one-analysis.sh`, and `examples/customer-settings/README.md`.